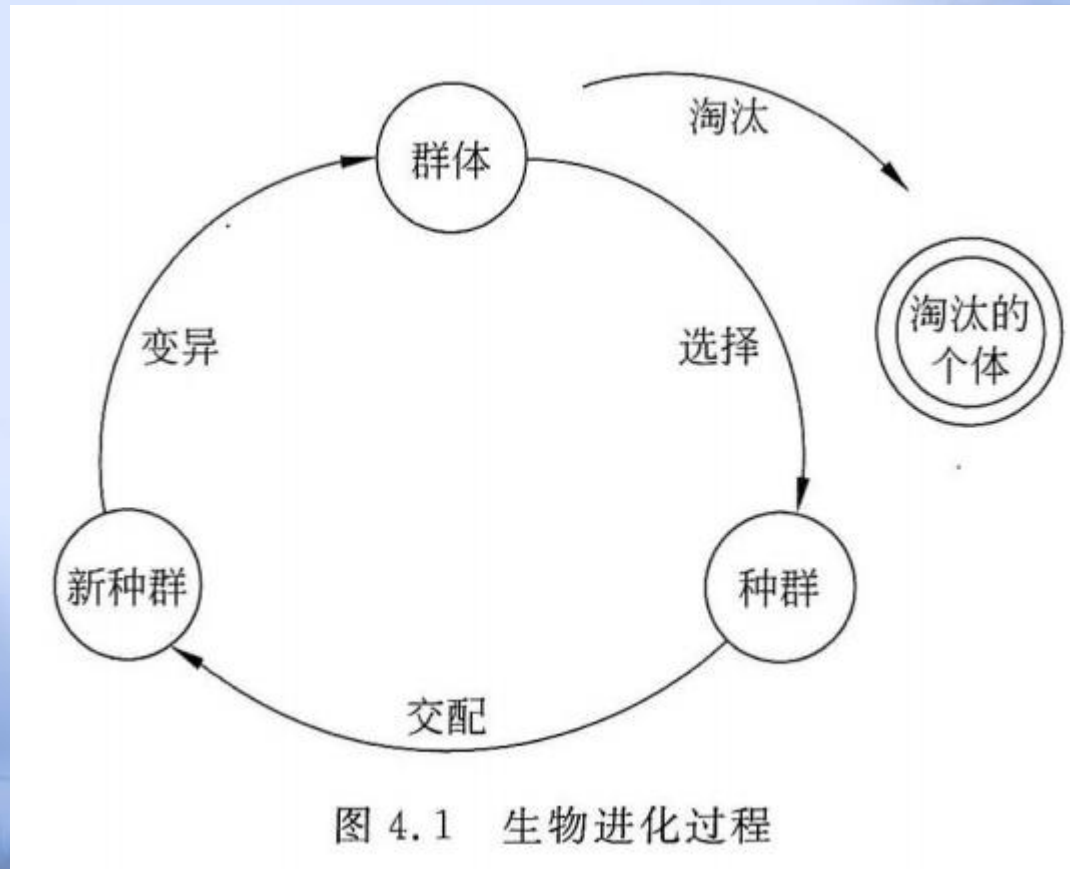


遗传算法

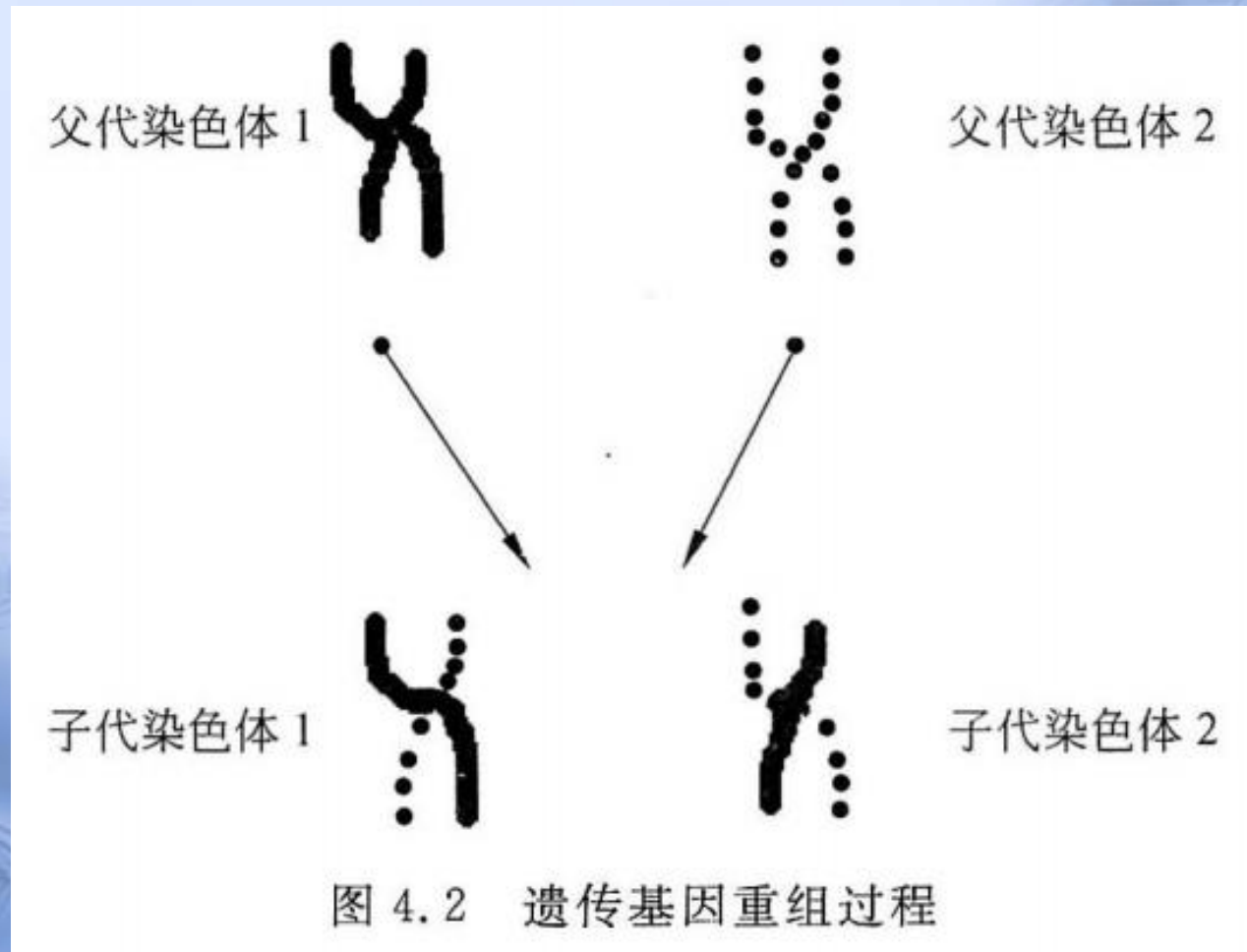
(Genetic Algorithm, GA)

了解遗传算法的研究背景，熟练掌握遗传算法的思想来源和设计流程，掌握遗传算法的参数设计和影响作用，并能理解遗传算法的改进和实际应用。

"自然选择"和"优胜劣汰"的进化规律。



遗传信息的重组



遗传算法简介

遗传算法是模仿生物遗传学和自然选择机理，通过人工方式构造的一类优化搜索算法，是对生物进化过程进行的一种数学仿真，是进化计算的一种重要的形式。遗传算法与传统数学模型截然不同，它为那些难以找到传统数学模型的难题找出了解决方法。同时，遗传算法借鉴了生物学中的某些知识，从而体现了人工智能这一交叉学科的特点。霍兰德(Holland)于1975年在他的著作“Adaptation in Natural and Artificial Systems”中首次提出遗传算法。

遗传算法原理

- 遗传算法通过模拟自然界中生物的遗传进化过程，对优化问题的最优解进行搜索。
- 算法维护一个代表问题潜在解的群体，通过对群体的进化，搜索问题的最优解，算法中引入了类似自然进化过程中的选择、交叉、变异等算子。
- 遗传算法搜索全局最优解的过程是一个不断迭代的过程(每一次迭代相当于生物进化中的一次循环)，直到满足算法的终止条件为止。

遗传算法原理

- 在遗传算法中，问题的每个有效解被称为一个“**染色体(chromosome)**”，也称为“串”，对应于群体中的每个生物个体(individual) 。
- 染色体的具体形式是一个使用特定编码方式生成的编码串，编码串中的每一个编码单元称为"**基因(gene)**"
- 遗传算法通过比较**适应值(fitness value)** 区分染色体的优劣，适应值越大的染色体越优秀。
- **评估函数(evaluation function)** 用来计算并确定染色体对应的适应值。

遗传算法原理

- **选择算子(selection)** 按照一定的规则对群体的染色体进行选择，得到父代种群。一般情况下，越优秀的染色体被选中的次数越多。
- **交叉算子(crossover)** 作用于每两个成功交配的父代染色体，染色体交换各自的部分基因，产生两个子代染色体。子代染色体取代父代染色体进入新种群，而没有参与交叉运算的染色体则直接进入新种群。
- **变异算子(mutation)** 使新种群进行小概率的变异。染色体发生变异的基因改变数值，得到新的染色体。经过变异的新种群替代原有群体进入下一次进化。

遗传算法原理

表 4.1 生物遗传进化的基本生物要素和遗传算法的基本要素定义对照表

生物遗传进化	遗传算法
群体	问题搜索空间的一组有效解(表现为群体规模 N)
种群	经过选择产生的新群体(规模同样为 N)
染色体	问题有效解的编码串
基因	染色体的一个编码单元
适应能力	染色体的适应值
交配	两个染色体交换部分基因得到两个新的子代染色体
变异	染色体某些基因的数值发生改变
进化结束	算法满足终止条件时结束,输出全局最优解

遗传算法原理

遗传算法类似于自然进化，**通过作用于染色体上的基因寻找好的染色体来求解问题**。遗传算法对求解问题的本身一无所知，它所需要的仅仅是对算法所产生的每个染色体进行评价，并基于适应值来选择染色体，使适应性好的染色体有更多的繁殖机会。

在遗传算法中，通过随机方式产生若干个所求解问题的数字编码，即染色体，形成**初始种群**；通过适应度函数给每个个体一个**数值评价**，淘汰低适应度的个体，选择高适应度的个体参加**遗传操作**，经过遗传操作后的个体集合形成下一代**新的种群**。再对这个新种群进行下一轮进化。**这就是遗传算法的基本原理。**

遗传算法的流程

GA的算法在解空间中取一群点，作为遗传开始的第一代。每个点用一个编码数字串表示，其优劣程度用一个目标函数—适应度函数(**fitness function**)来衡量。

- 染色体的编码;
- 群体的初始化;
- 适应值评价;
 - 种群选择;
 - 种群交叉;
 - 种群变异;
- 算法流程

遗传算法的流程

1. 染色体的编码

许多应用问题的结构很复杂，我们希望找到一种既简单又不影响算法性能的编码方式。**将问题结构变换为位串形式编码表示的过程叫做编码**；相反的，将位串形式编码表示变换为原问题结构的过程叫做解码或译码。

关于确定遗传算法染色体编码方式的两条指导原则：**有意义积木块编码**原则和**最小字符集编码**原则，倡导算法使用的编码方案应易于产生低阶且定义长度较短的模式，在能够自然描述所求问题的前提下使用最小编码字符集

遗传算法的流程

1. 二进制编码方法

二进制编码方法产生的染色体是一个二进制符号序列，染色体的每一个基因只能取值 **0** 或 **1**。

表 4.2 染色体的二进制编码方法

二进制符号串	对应的实际取值
0000...0000	$U_{\min}$
1111...1111	$U_{\max}$
$X_L X_{L-1} \cdots X_2 X_1$	$U_{\min} + \frac{(U_{\max} - U_{\min}) \sum_{j=1}^L X_j 2^{j-1}}{2^L - 1}$

假定问题定义的有效解**取值空间**为 $[U_{\min}, U_{\max}]$ ，其中 D 为有效解的**变量维数**，使用 L 位二进制符号串表示解的一维变量，则我们可以得到如表 4.2 所示的编码方式：

2. 浮点数编码方法

- 浮点数编码方法中，每个染色体用某一范围内的一个**浮点数来表示**，染色体的编码长度等于问题定义的解的变量个数，染色体的**每一个基因等于解的每一维变量**。

待求解问题的一个有效解为 $X_i = (x_i^1, x_i^2, \dots, x_i^{D-1}, x_i^D)$

则该解对应的染色体编码为 $(x_i^1, x_i^2, \dots, x_i^{D-1}, x_i^D)$

- 因为这种编码方法使用的是变量的真实值，所以浮点数编码方法也叫做真值编码方法。对于一些多维、高精度要求的连续函数优化问题，用浮点数编码来表示个体时将会有一些益处。

遗传算法的流程

2. 群体的初始化

- 遗传算法在给定的初始群体中进行迭代搜索
- 采用生成随机数的方法，对染色体的每一维变量进行初始化赋值
- 初始化染色体时必须满足优化问题对**有效解**的定义。
- 在算法的开始得到一个平均适应值相对较高的初始群体再进行进化来提高算法的求解性能

3. 适应值评价：用于评估各个染色体的适应值，区分优劣

- 为了体现染色体的适应能力，引入了对每一个染色体都能进行度量的函数，叫做**适应度函数** (fitness function)。通过计算**适应度函数值**来决定染色体的优劣程度，它体现了自然进化中的优胜劣汰原则。
- 在遗传算法中，规定适应值越大的染色体越优。适应度函数要求能有效地反映每一个染色体的适应能力。若一个染色体与问题的最优解染色体之间的差距较小，则对应的适应度函数值就会较大。

- 评估函数常常 **根据问题的优化目标** 来确定，例如求解函数优化问题时，问题定义的目标函数可以作为评估函数的原型。
- 对于一些求解最大值的数值优化问题，我们可以直接套用问题定义的函数表达式。但是对于其他优化问题，问题定义的目标函数表达式必须 **经过一定的变换**。例如TSP的目标是路径总长度为最短，路径总长度变换后就可作为TSP问题的适应度函数：

$$f(w_1 w_2 \cdots w_n) = \frac{1}{\sum_{j=1}^n d(w_j, w_{j+1})}$$

$d(w_j, w_{j+1})$ 表示两城市间的距离(路径长度)。

遗传操作：遗传算法的遗传操作主要有三种：**选择(selection)**、**交叉(crossover)**、**变异(mutation)**。

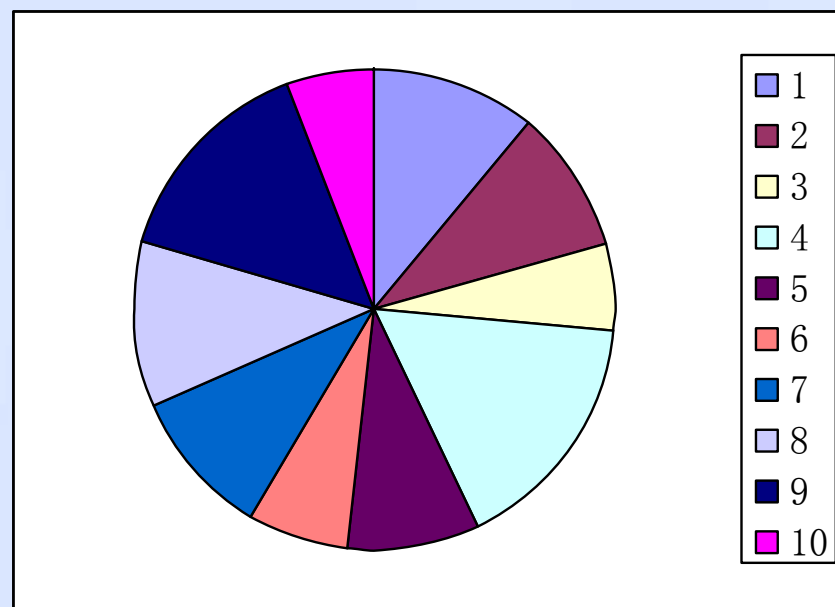
4. 选择算子：

选择操作也叫做复制(reproduction)操作，根据个体的适应度函数值所度量的优劣程度决定它在下一代是被淘汰还是被遗传。

一般地，适应度较大(优良)的个体有较大的存在机会，而适应度较小(低劣)的个体继续存在的机会也较小。简单遗传算法采用赌轮选择机制。

- 根据每个染色体的适应值得到群体中所有染色体的适应值总和。
- 计算每个染色体适应值与群体适应值总和的比 P_i

假设一个具有 N 个扇区的轮盘，*每个扇区对应群体中的一个染色体*，扇区的大小与对应染色体的 P_i 值成正比。



5. 交叉算子:

- 交叉操作的简单方式是: 按交叉概率 P_c 选择出两个父代个体 P_1 和 P_2 , 将两者的部分基因(码值)进行交换。

- 交叉概率的 P_c 一般取值为: 0.4~0.99之间。
- 随机数 $\text{Random}(0, 1)$ 小于 P_c , 则表示该染色体可进行交叉操作
- 随机产生一个有效的交配位置, 父代染色体交换位于该交配位置后的所有基因

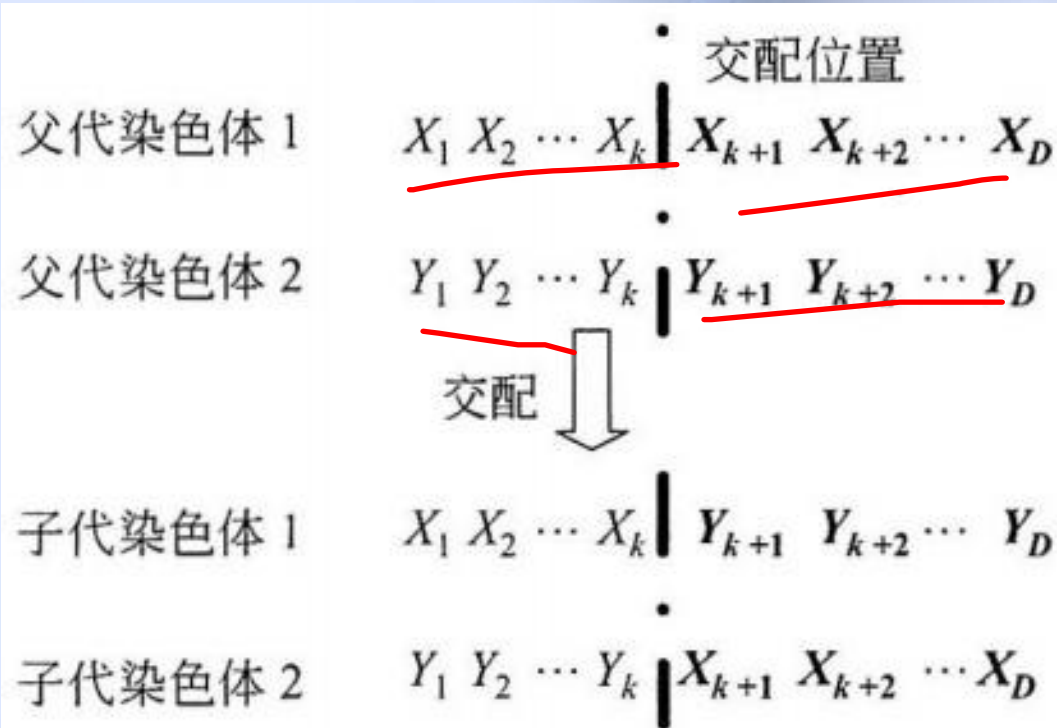


图 4.7 染色体交配示意图

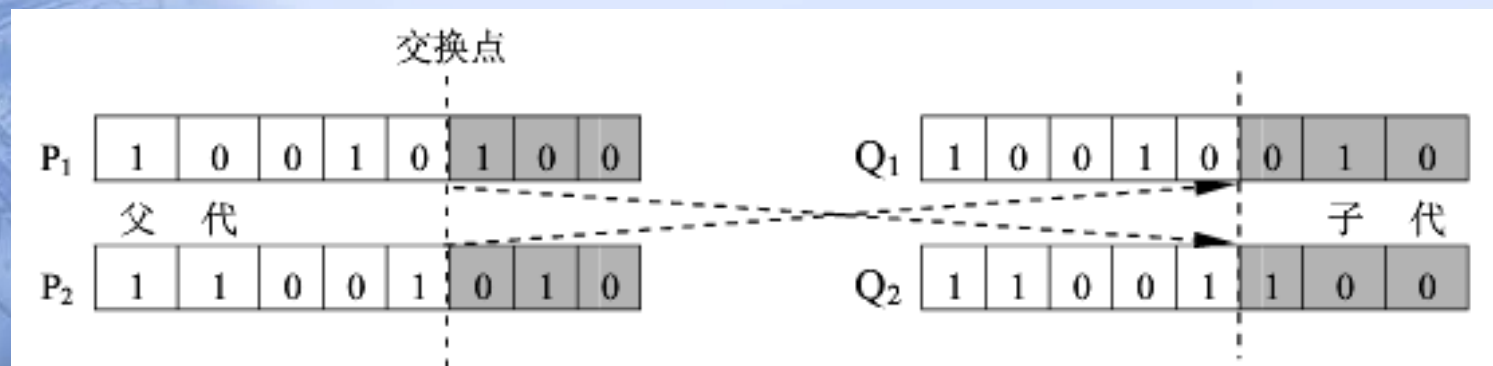
P_1

1	0	0	1	0	1	0	0
---	---	---	---	---	---	---	---

 P_2

1	1	0	0	1	0	1	0
---	---	---	---	---	---	---	---

编码长度为8，产生一个在1~7之间的随机数 k ，假如现在产生的是5，将 P_1 和 P_2 的后3位基因交换： P_1 的高5位与 P_2 的低三位组成数串10001001，这就是 P_1 和 P_2 的一个子代个体 Q_1 ； P_2 的高5位与 P_1 的低3位组成数串11011110，这就是 P_1 和 P_2 的另一个子代 Q_2 个体。交换过程如图所示



2) 双点交叉

它的具体操作过程是：(1) 在相互配对的两个个体编码串中随机设置两个交叉点；(2) 交换两个交叉点之间的部分基因。当 $p_1 = (a_{L-1}a_{L-2} \cdots a_0)$ 是固定的，另外一个父体 $p_2 = (b_{L-1}b_{L-2} \cdots b_0)$ 随机均匀选取时，使用双点均匀交叉生成的子代个体 $C_1 = (a_{L-1}a_{L-2} \cdots a_k b_{k-1} \cdots b_j a_{j-1} \cdots a_0)$ ，两交叉点的距离为 $k - j$ 。

3) 均匀交叉

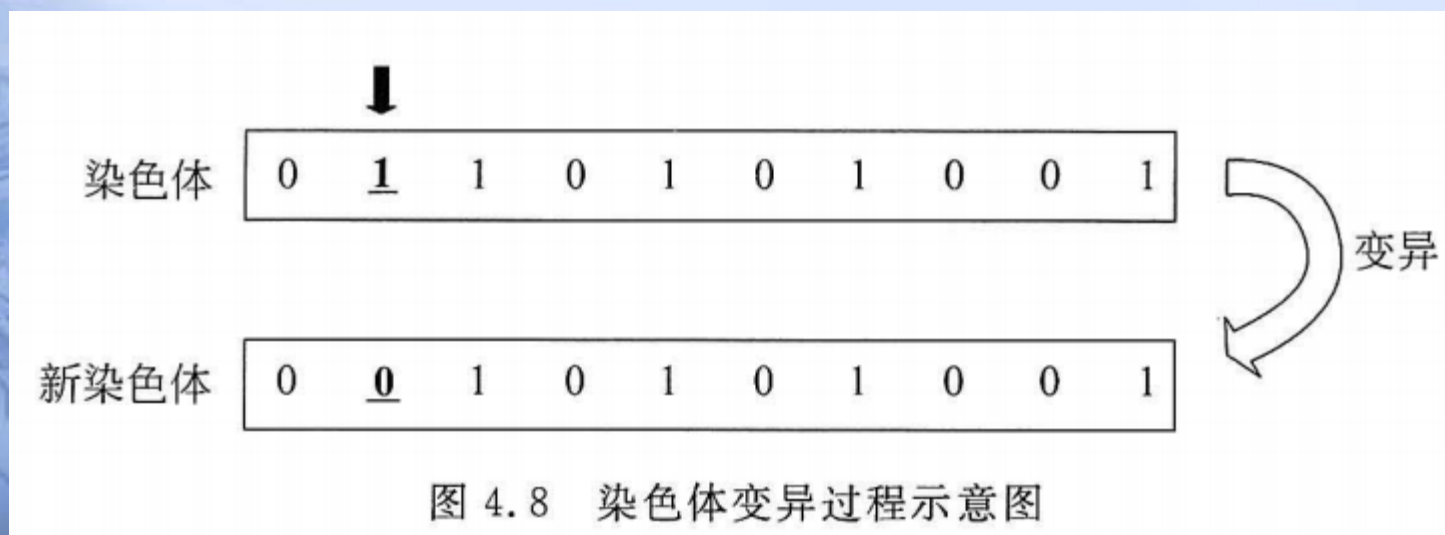
它是指两个配对个体的每一位基因都以相同的概率进行交换，从而形成两个新个体。具体操作过程如下：

(1) 随机产生一个与个体字符串长度相同的二进制随机数 $W = w_{L-1}w_{L-2} \cdots w_0$ ；

(2) 按下列规则从 A 、 B 两个父代个体中产生两个新个体 X 、 Y ：若 $w_i = 0$ ，则 X 的第 i 个字符继承 A 的对应字符， Y 的第 i 个字符继承 B 的对应字符；若 $w_i = 1$ ，则 A 、 B 的第 i 个字符相互交换，从而生成 X 、 Y 的第 i 个字符，

6. 变异算子：变异操作的简单方式就是改变码串中某个位置上的数码

- 变异概率的 P_m 一般取值为：0.001~0.1之间。
- 随机数 $\text{Random}(0, 1)$ 小于 P_m ，则表示该染色体可进行变异操作
- 随机产生一个有效的变异位置，染色体上位于该位置的基因发生改变



- 二进制编码表示的每个位置的数码只有0和1两种可能，二进制编码的简单变异操作是将0与1互换：0变异为1，1变异为0。设有如下二进制编码：

1	0	1	0	0	1	1	0
---	---	---	---	---	---	---	---

- 码长为8，随机产生一个1~8之间的数 k ，假如现在 $k=5$ ，对第5位(从右到左)进行变异操作，将原来的0变为1，得到如下数码串：

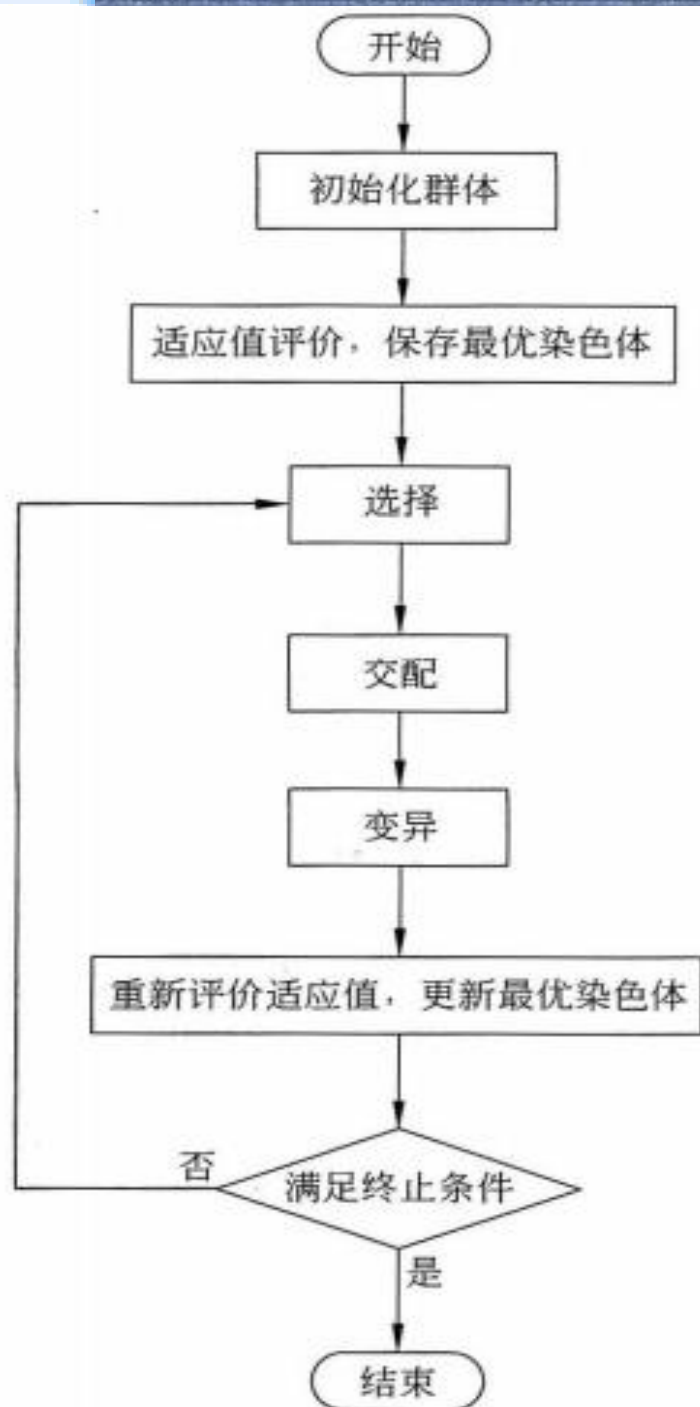
1	0	1	1	0	1	1	0
---	---	---	---	---	---	---	---

- 浮点数编码形式的染色体若某基因发生变异，则可采用随机数方法随机产生一个满足问题定义的数值取代该基因现有的值。

7. 算法流程：一般遗传算法的主要步骤如下

- **Step 1 初始化** 规模为 N 的群体，其中染色体每个基因的值采用随机数产生器生成并满足问题定义的范围。当前进化代数 $\text{Generation} = 0$ 。
- **Step 2** 用评估函数对群体中所有染色体进行评价，分别计算每个染色体的**适应值**，保存适应值最大的染色体 **Best**
- **Step 3** 采用轮盘赌**选择运算**对群体的染色体进行选择操作，产生规模同样为 N 的种群。
- **Step 4** 按照概率 P_c 从种群中选择染色体进行**交叉运算**。两两父代染色体交换部分基因，产生两个新的子代染色体，子代染色体取代父代染色体进入新种群。没有进行交叉的染色体直接复制进入新种群。

- Step 5 按照概率 P_m 对新种群中染色体的基因进行变异操作。发生变异的基因数值发生改变。变异后的染色体取代原有染色体进入新群体，未发生变异的染色体直接进入新群体。
- Step 6 变异后的新群体取代原有群体，重新计算群体中各个染色体的适应值。倘若群体的最大适应值大于 Best 的适应值，则以该最大适应值对应的染色体替代 Best，即更新最大适应值大于 Best。
- Step 7 当前进化代数 Generation 加 1。如果 Generation 超过规定的最大进化代数或 Best 达到规定的误差要求，算法结束，Best 可表示问题的一个解；否则返回 Step 3



/*

$P(t)$ 表示某一代的群体, t 为当前进化代数

Best 表示目前已找到的最优解

*/

Procedure GA

begin

$t \leftarrow 0$;

initialize($P(t)$); // 初始化群体

evaluate($P(t)$); // 适应值评价

keep_best($P(t)$); // 保存最优染色体

while (不满足终止条件) **do**

begin

$P(t) \leftarrow \text{selection}(P(t))$; // 选择算子

$P(t) \leftarrow \text{crossover}(P(t))$; // 交配算子

$P(t) \leftarrow \text{mutation}(P(t))$; // 变异算子

$t \leftarrow t + 1$;

$P(t) \leftarrow P(t-1)$;

evaluate($P(t)$);

if ($P(t)$ 的最优适应值大于 Best 的适应值)

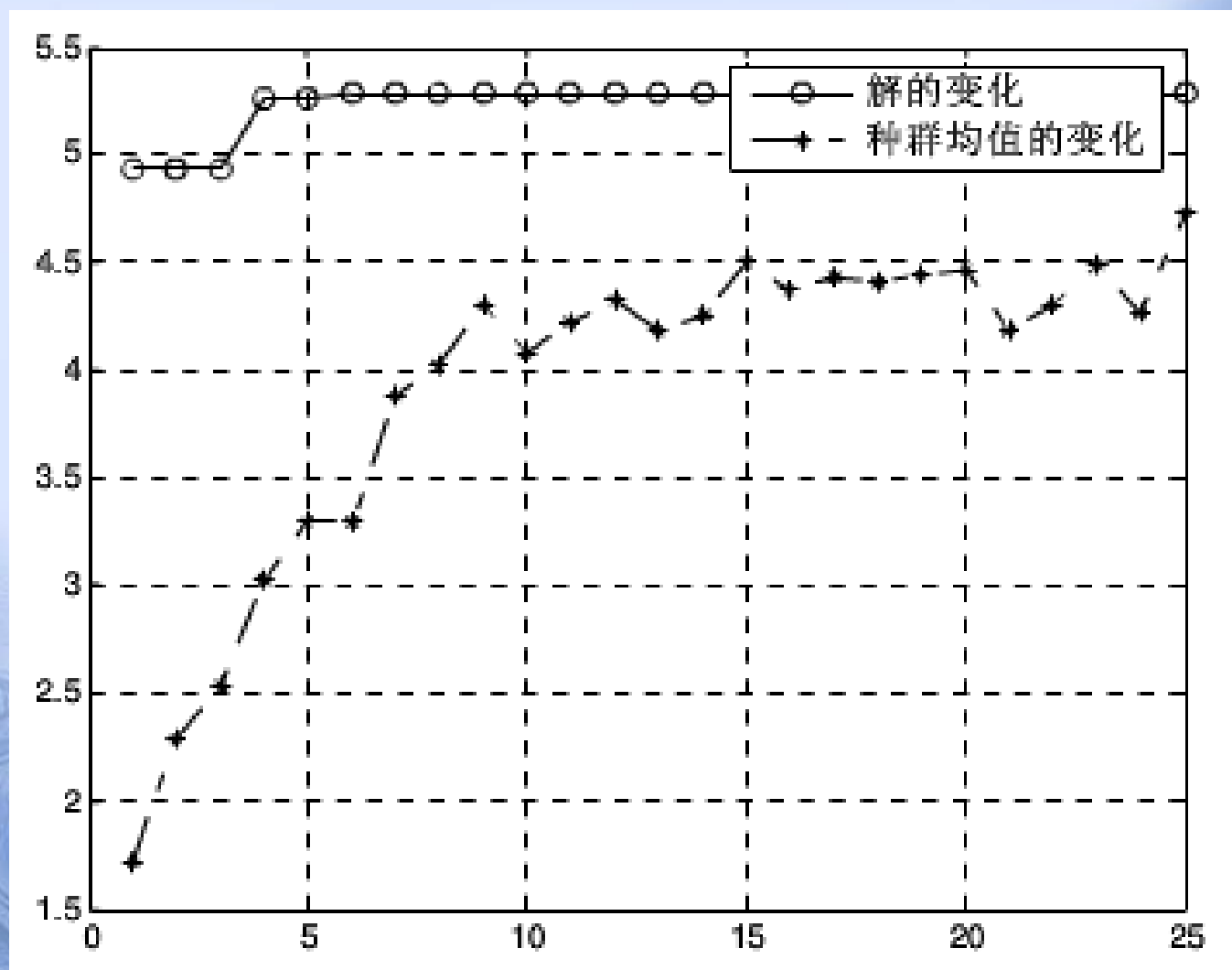
// 以 $P(t)$ 的最优染色体替代 Best

replace(Best);

end if

end

end



遗传算法的参数设置

群体规模 N

- ✓ 影响算法的搜索能力和运行效率。
- 若 N 设置较大，一次进化所覆盖的模式较多，可以保证群体的多样性，从而提高算法的搜索能力，但是由于群体中染色体的个数较多，势必增加算法的计算量，降低算法的运行效率。
- 若 N 设置较小，虽然降低了计算量，但是同时降低了每次进化中群体包含更多较好染色体的能力。
- N 的设置一般为 20~100 。

染色体的长度 L

√影响算法的计算量和交叉变异操作的效果。

- L 的设置跟优化问题密切相关，一般由问题定义的解的形式和选择的编码方法决定。
- 对于二进制编码方法，染色体的长度 L 根据解的取值范围和规定精度要求选择大小。
- 对于浮点数编码方法，染色体的长度 L 跟问题定义的解空间维数 D 相同。
- 除了染色体长度一定的编码方法，Goldberg 等人还提出了一种变长度染色体遗传算法 Messy GA，其染色体的长度并不是固定的。

基因的取值范围 R

✓ R 视采用的染色体编码方案而定。

- 对于二进制编码方法， $R = \{0, 1\}$
- 对于浮点数编码方法， R 与优化问题定义的解每一维变量的取值范围相同。

交叉概率 P_c

- ✓ 决定了进化过程中，种群参加交叉运算的染色体的平均数目 $P_c \times N$ 。
- P_c 的取值范围一般为 **0.4~0.99**
- 也可采用自适应的方法调整算法运行过程中的 P_c 值

变异概率 P_m

- ✓ 增加群体进化的多样性，决定了进化过程中群体发生变异的基因平均个数。
- P_m 的值不宜过大。因为变异对已找到的较优解具有一定的破坏作用，如果 P_m 的值太大，可能会导致算法目前所处的较好的搜索状态倒退回原来较差的情况。
- P_m 的取值一般为 0.001~0.1
- 也可采用自适应的方法调整算法运行过程中的 P_m 值

适应值评价

- √ 影响算法对种群的选择，恰当的评估函数应该能够对染色体的优劣做出合适的区分，保证选择机制的有效性，从而提高群体的进化能力。
- 评估函数的设置同优化问题的求解目标有关。
- 评估函数应满足较优染色体的适应值较大的规定。
- 为了更好地提高选择的效能，可以对评估函数做出一定的修正。
- 目前主要的评估函数修正方法有：
 - 线性变换
 - 乘幂变换
 - 指数变换等

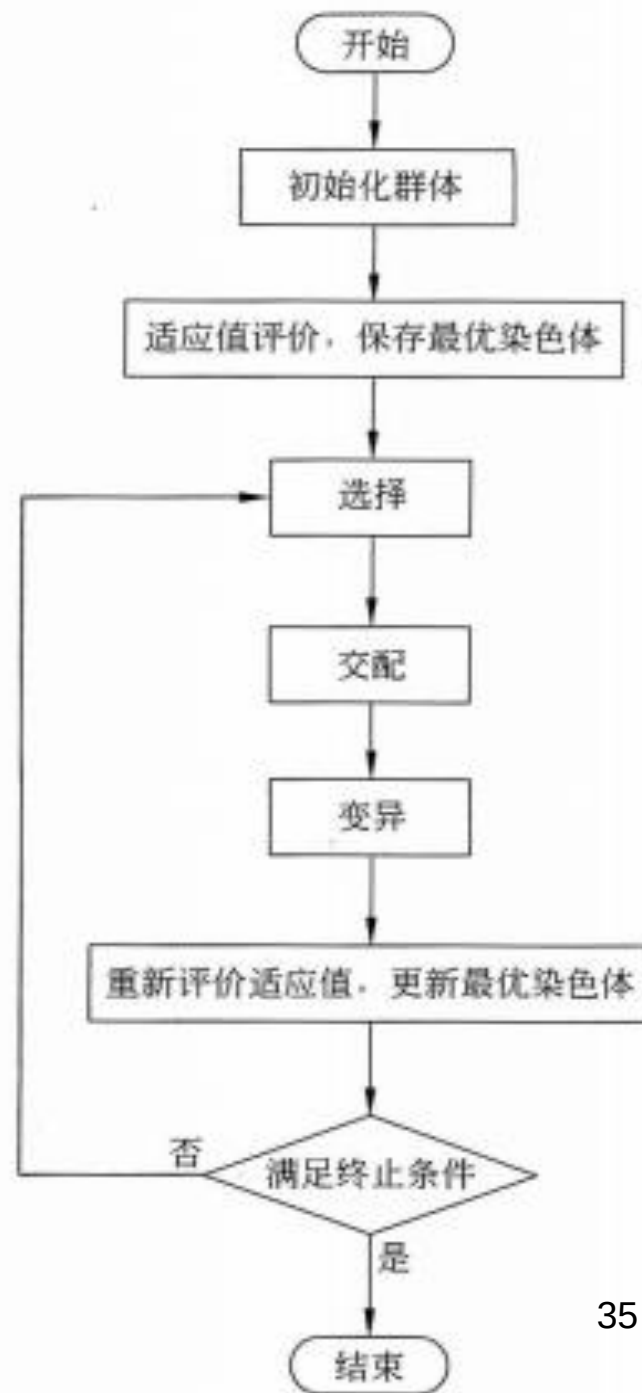
终止条件

- √ 决定算法何时停止运行，输出找到的最优解。
- 采用何种终止条件，跟具体问题的应用有关。
- 可以使算法在**达到最大进化代数**时停止，最大进化代数一般可设置为 **100~1000**，根据具体问题可对该建议值作相应的修改。
- 也可以通过考察**找到的当前最优解**的情况来控制算法的停止。
- 例如，当目前进化过程算法找到的最优解达到一定的误差要求，则算法可以停止。误差范围的设置同样跟具体的优化问题相关。或者是算法在持续很长的一段进化时间内所找到的最优解没有得到改善时，算法可以停止。

例 4.1 已知函数

$$y = f(x_1, x_2, x_3, x_4) \\ = \frac{1}{x_1^2 + x_2^2 + x_3^2 + x_4^2 + 1},$$

其中 $-5 \leq x_1, x_2, x_3, x_4 \leq 5$,
用遗传算法求解 y 的最大值。



步骤 1：初始化

假设群体规模为 5；使用浮点数编码方式构造染色体，即每个染色体以 (x_1, x_2, x_3, x_4) 的形式表示。

初始化群体的染色体，得到

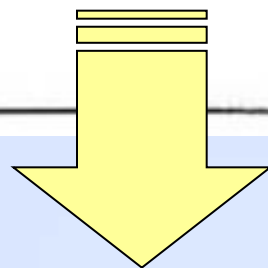
$$C_1 = (-2.1351, 2.0917, -0.1327, -4.1006)$$

$$C_2 = (1.0152, -3.9811, -2.6638, 3.7535)$$

$$C_3 = (4.0589, 2.1904, -0.1503, 0.0023)$$

$$C_4 = (-3.4098, -3.0714, -0.9008, -4.3712)$$

$$C_5 = (0.2073, 2.9932, -4.0802, 1.8794)$$



步骤 2：适应值评价
选择评估函数

$$Eval(C) = y = f(x_1, x_2, x_3, x_4) = \frac{1}{x_1^2 + x_2^2 + x_3^2 + x_4^2 + 1}$$

计算每个染色体的适应值如下：

$$Eval(C_1) = f(-2.1351, 2.0917, -0.1327, -4.1006) = 0.0373603$$

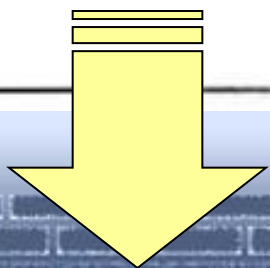
$$Eval(C_2) = f(1.0152, -3.9811, -2.6638, 3.7535) = 0.0255988$$

$$Eval(C_3) = f(4.0589, 2.1904, -0.1503, 0.0023) = 0.0448529$$

$$Eval(C_4) = f(-3.4098, -3.0714, -0.9008, -4.3712) = 0.0238214$$

$$Eval(C_5) = f(0.2073, 2.9932, -4.0802, 1.8794) = 0.0331319$$

因此 $Best = C_3$, $Eval(Best) = 0.0448529$





步骤 3：选择

采用轮盘赌选择算法, 计算群体适应值总和为 $0.0373603 + 0.0255988 + 0.0448529 + 0.0238214 + 0.0331319 = 0.164765$

分别计算每个染色体适应值同群体适应值总和的比:

$$C_1: 0.226749$$

$$C_2: 0.155365$$

$$C_3: 0.272223$$

$$C_4: 0.144578$$

$$C_5: 0.201085$$

下面是 5 次选择产生的 $[0, 1]$ 的随机数和选中的染色体:

$$(1) \quad 0.278756 \quad C_2$$

$$(2) \quad 0.604389 \quad C_3$$

$$(3) \quad 0.230964 \quad C_2$$

$$(4) \quad 0.376263 \quad C_2$$

$$(5) \quad 0.858791 \quad C_5$$

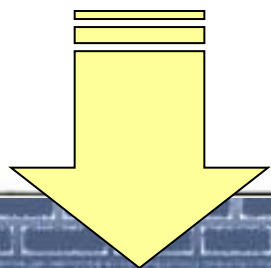
因此得到种群为:

$$C_1' = (1.0152, -3.9811, -2.6638, 3.7535)$$

$$C_2' = (4.0589, 2.1904, -0.1503, 0.0023)$$

$$C_3' = (1.0152, -3.9811, -2.6638, 3.7535)$$

$$C_4' = (1.0152, -3.9811, -2.6638, 3.7535)$$

$$C_5' = (0.2073, 2.9932, -4.0802, 1.8794)$$


步骤4：交配

假设交配概率为 0.88。下面是对每个染色体生成的 $[0,1]$ 的随机数，决定染色体是否参加交配。

C_1' 0.341044 < 0.88 参加交配

C_2' 0.6137971 < 0.88 参加交配

C_3' 0.963042 > 0.88 不参加交配

C_4' 0.347545 < 0.88 参加交配

C_5' 0.593677 < 0.88 参加交配

即 C_1' 和 C_2' 、 C_4' 和 C_5' 进行交配，每对染色体交配时随机生成 0~3 之间的自然数作为交配位。以下是各对染色体的交配位和得到的子代染色体。

(1) $C_1' = (1.0152, -3.9811, -2.6638, 3.7535)$

和 $C_2' = (4.0589, 2.1904, -0.1503, 0.0023)$

交配位为 1; 子代染色体为：

$C_1'' = (1.0152, -3.9811, -0.1503, 0.0023)$

$C_2'' = (4.0589, 2.1904, -2.6638, 3.7535)$

(2) $C_4' = (1.0152, -3.9811, -2.6638, 3.7535)$

和 $C_5' = (0.2073, 2.9932, -4.0802, 1.8794)$

交配位为 2; 子代染色体为：

$C_4''' = (1.0152, -3.9811, -2.6638, 1.8794)$

$C_5''' = (0.2073, 2.9932, -4.0802, 3.7535)$

故交配后的新种群为：

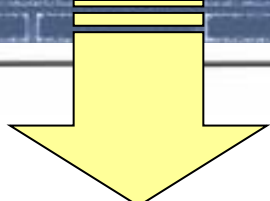
$C_1''' = (1.0152, -3.9811, -0.1503, 0.0023)$

$C_2''' = (4.0589, 2.1904, -2.6638, 3.7535)$

$C_3''' = (1.0152, -3.9811, -2.6638, 3.7535)$

$C_4''' = (1.0152, -3.9811, -2.6638, 1.8794)$

$C_5''' = (0.2073, 2.9932, -4.0802, 3.7535)$



步骤 5：变异

假设变异概率为 0.1。对于每个染色体的每个基因随机生成 [0,1] 的随机数,若该随机数小于 0.1,则改变基因的值,否则不改变基因的值。以下是发生变异的染色体和基因改变的过程。

$$C_3''' = (\underline{1.0152}, -3.9811, -2.6638, 3.7535) \rightarrow$$

$$C_3''' = (\underline{3.0953}, -3.9811, -2.6638, 3.7535)$$

$$C_4''' = (1.0152, \underline{-3.9811}, -2.6638, 1.8794) \rightarrow$$

$$C_4''' = (1.0152, \underline{0.0153}, -2.6638, 1.8794)$$

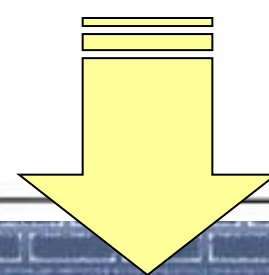
故得到的新群体为：

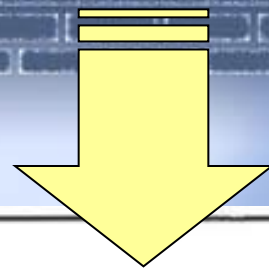
$$C_1''' = (1.0152, -3.9811, -0.1503, 0.0023)$$

$$C_2''' = (4.0589, 2.1904, -2.6638, 3.7535)$$

$$C_3''' = (3.0953, -3.9811, -2.6638, 3.7535)$$

$$C_4''' = (1.0152, 0.0153, -2.6638, 1.8794)$$

$$C_5''' = (0.2073, 2.9932, -4.0802, 3.7535)$$




步骤 6：重新评价染色体适应值，更新 *Best*
计算每个染色体的适应值如下：

$$Eval(C_1''') = f(1.0152, -3.9811, -0.1503, 0.0023) = 0.0558585$$

$$Eval(C_2''') = f(4.0589, 2.1904, -2.6638, 3.7535) = 0.0230112$$

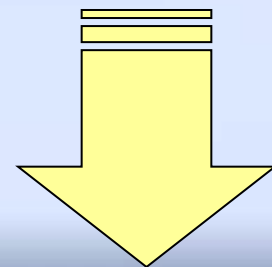
$$Eval(C_3''') = f(3.0953, -3.9811, -2.6638, 3.7535) = 0.0210019$$

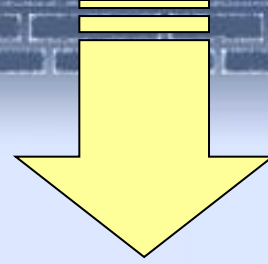
$$Eval(C_4''') = f(1.0152, 0.0153, -2.6638, 1.8794) = 0.0789962$$

$$Eval(C_5''') = f(0.2073, 2.9932, -4.0802, 3.7535) = 0.0245465$$

因为 $Max(0.0558585, 0.0230112, 0.0210019, 0.0789962, 0.0245465) = 0.0789962 > Eval(Best)$ ，故更新 *Best*：

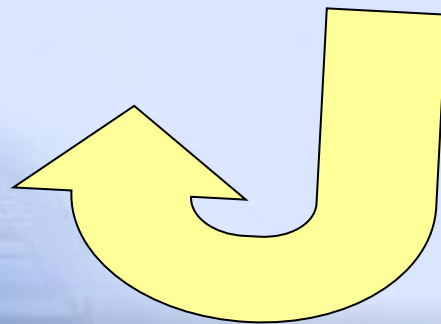
$Best = C_4'''$ ， $Eval(Best) = 0.0789962$ 。





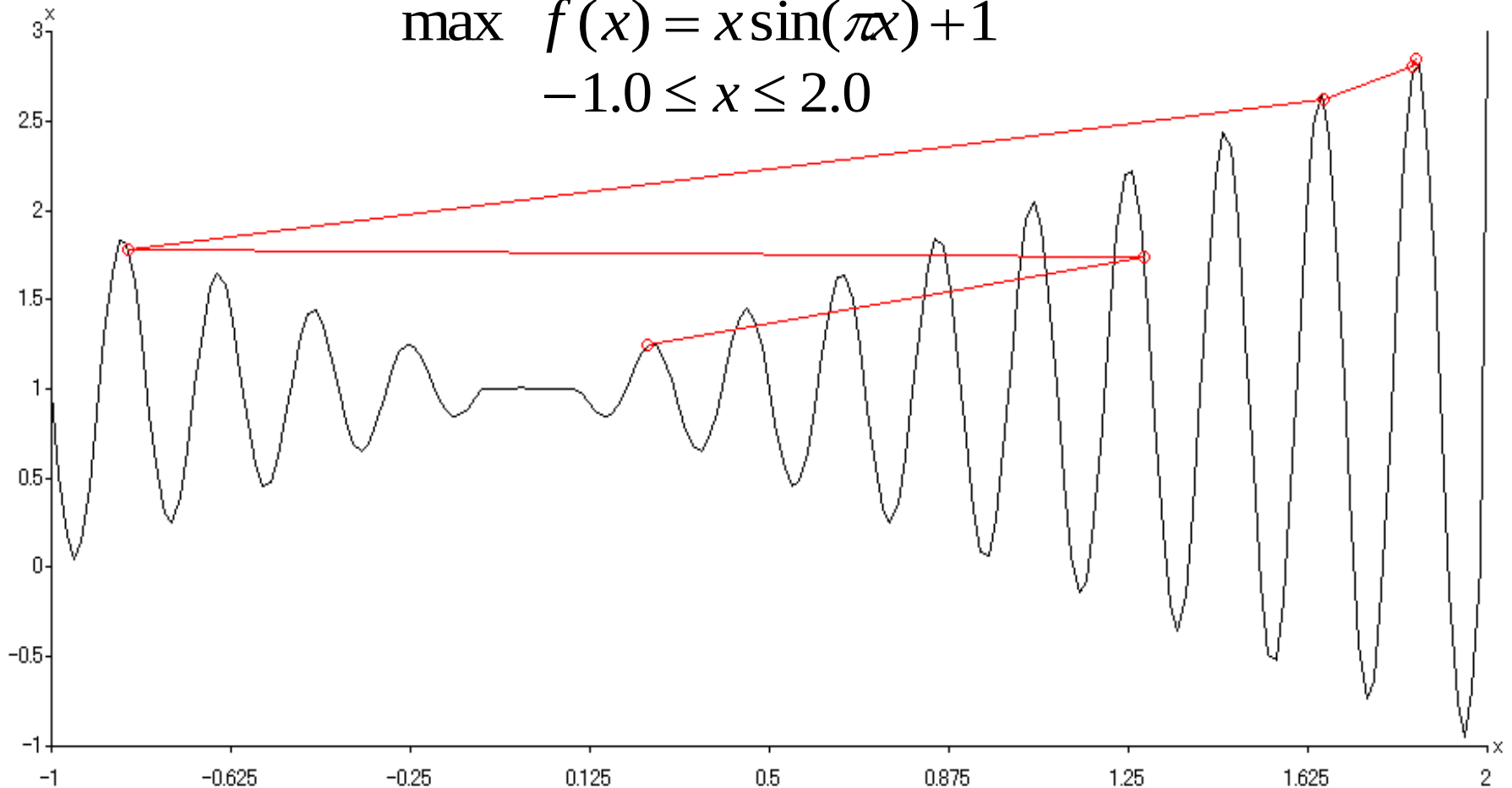
步骤 7：判断结束

如果满足算法终止条件，则输出找到的最优解 *Best* 并退出程序；否则返回步骤 3 继续执行。



Example of Genetic Algorithm for Unconstrained Numerical Optimization (Michalewicz, 1996)

$$\max f(x) = x \sin(\pi x) + 1$$
$$-1.0 \leq x \leq 2.0$$



遗传算法解决TSP问题

在 n 个城市的 TSP 问题中,一条有效的路径可以看成 n 个城市的一种排列. n 个城市有 $n!$ 种排列. 注意到两个顺序完全相反的方案其行程相同,而对于一种排列从哪个城市出发都可以,所以有效路径的方案数目为 $R_n = \frac{n!}{2n}$. 例如, $R_4 = 3, R_5 = 12, R_6 = 120, R_{10} = 181440, R_{30} \approx 4.4 \times 10^{30}, R_{60} \approx 6.93 \times 10^{78}, R_{100} \approx 4.67 \times 10^{155}$. 可见路径总数随 n 增大急剧增长,当城市数目增加,计算量增加到无法进行的地步.

若用穷举搜索算法,则须考虑所有可能的情况,找出所有的路径,再对其进行比较,来找到最佳路径. 这种方法随着城市数 n 的上升,算法时间随 n 按指数规律增长,即存在所谓的指数爆炸(也称组合爆炸)问题.

遗传算法解决TSP问题

(1) 编码和初始种群

在求解 TSP 问题的遗传算法中,多采用以遍历城市的次序排列进行编码的方法. 如码串 123456789 表示自城市 C_1 开始,依次经城市 $C_2, C_3, \dots, C_9$ 最后返回城市 C_1 的遍历路径. 这是一种针对 TSP 问题的最自然的编码方式,不过这一编码方案引起了交叉操作的困难,所以应在交叉操作时设计特殊的适合于处理 TSP 问题的交叉策略.

遗传算法解决TSP问题

(2) 适应度函数的定义

TSP 问题是寻找 n 个城市的排列,使有效路径长度

$$T_d = \sum_{i=1}^{n-1} d_{i,i+1} + d_{n,1}$$

取最小值,式中 d_{ij} 表示城市 C_i 到城市 C_j 的距离.

在利用遗传算法求解 TSP 问题中,

适应度函数常取路径长度的倒数,即 $f = \frac{1}{T_d}$.

遗传算法解决TSP问题

(3) 选择和交叉操作

对于 TSP 问题,基于上述顺序编码,若用单点交叉或多点交叉策略,必然以极大的概率导致未能完全遍历所有城市的非法路径,如 TSP 问题的两个父本路径为

1234 | 56789

9876 | 54321

若用单点交叉,且交叉点随机选为 4,则交叉后产生的两个后代为

123454321

987656789

(3) 交叉操作

1. 部分匹配交叉 (Partially Matched Crossover, PMX) 法

例如, 设两父串及匹配区域为

$$A = 91 \mid 456 \mid 7832$$

$$B = 68 \mid 123 \mid 9547$$

首先交换 A 和 B 的两个匹配区域, 得到

$$A' = 91 \mid 123 \mid 7832$$

$$B' = 68 \mid 456 \mid 9547$$

然后, 对于 A' 、 B' 两子串中匹配区域以外出现的遍历重复进行交换.

对于 A' 有 $1 \rightarrow 4, 2 \rightarrow 5, 3 \rightarrow 6$, 于是得

$$A'' = 941237865$$

$$B'' = 384569217$$

(3) 交叉操作

2. 顺序交叉 (Order Crossover, 简称 OX) 法

$$A = 91 \mid 456 \mid 7832$$

$$B = 68 \mid 123 \mid 9547$$

$$A' = 91 \mid 123 \mid 7832$$

$$B' = 68 \mid 456 \mid 9547$$

根据匹配区域的映射关系,在其区域外的相应位置标记 H ,得

$$A' = 9H \mid 456 \mid 78HH$$

$$B' = H8 \mid 123 \mid 9HH7$$

再移动匹配区,预留相等于匹配区域的空间(H 的数目)

$$A'' = 456HHH789$$

$$B'' = 123HHH978$$

将父串 A, B 的匹配区域相互交换,并放置到 A'', B'' 的预留区内,即可得到两个子代

$$A''' = 456123789$$

$$B''' = 123456789$$

(3) 交叉操作

一种改进的 OX 法如下：

设两父串及匹配区域为

$$A = 91 \mid 456 \mid 7832$$

$$B = 68 \mid 123 \mid 9547$$

首先将 A 的匹配区加到 B 的前面, B 的匹配区加到 A 的前面,

$$A' = 123914567832$$

$$B' = 456681239547$$

然后, 在 A' 和 B' 中分别在匹配区后依次删除与匹配区相同的城市码, 得最终子串为

$$A'' = 123945678$$

$$B'' = 456812397$$

(4) 变异操作

1. 逆转变异

在码串中随机选择两个逆转点,然后将这两点内的子串逆向排序. 例如,设串 A 的逆转点为 3,6,则经逆转变异后变为 A' ,有

$$A = 123 \mid 456 \mid 789$$

$$A' = 123 \mid 654 \mid 789$$

2. 对换变异

随机选择码串中两点作为对换点,对换其值. 如对于串 A ,选择对换点为 3,8,得

$$A = 12 \underline{3}4567 \underline{8}9$$

则经对换后变为 A' ,

$$A' = 12 \underline{8}4567 \underline{3}9$$

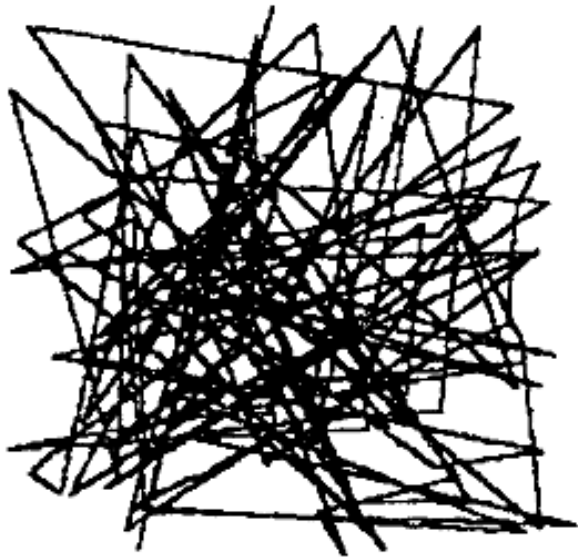
3. 插入变异

从码串中随机选择一个码,将此码插入随机选择的插入点中间. 例如对于上述串 A ,若取插入码为 6,选取插入点为 3~4 之间,则有

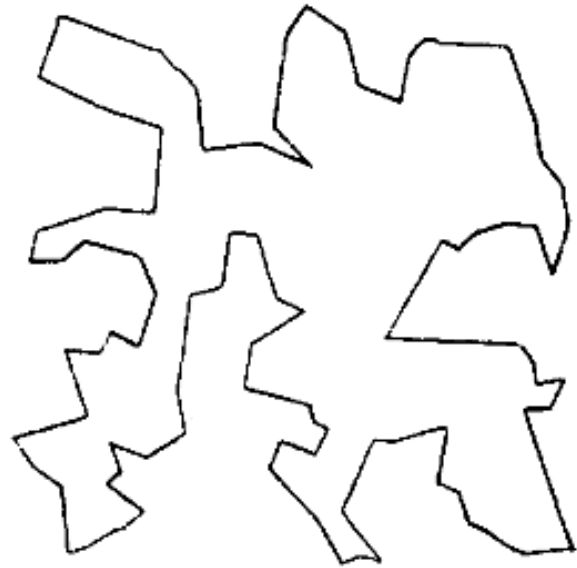
$$A' = 123645789$$

遗传算法解决TSP问题

(5) 优化过程



100 城市 TSP 问题的初始最佳路径($G=0$ 代)



100 城市 TSP 问题经 GA 优化后的最佳路径($G=200$ 代)

未成熟收敛

未成熟收敛是遗传算法中不可忽视的现象，它主要表现在以下两个方面：**群体中所有的个体都陷于同一极值而停止进化；接近最优解的个体总是被淘汰，进化过程不收敛**

在遗传算法的处理过程中每个环节都有可能导入产生未成熟收敛的因素，**具体表现**为：

- 在进化初始阶段，生成了具有较高适应度的个体 X 。
- 在基于适应度比例的选择下，其他个体被淘汰，大部分个体与 X 一致。
- 相同的两个个体实行交叉，从而未能生成新个体。
- 通过变异或逆转所生成的个体适应度高但数量少，所以被淘汰的概率很大。
- 群体中大部分个体都处于与 X 一致的状态。

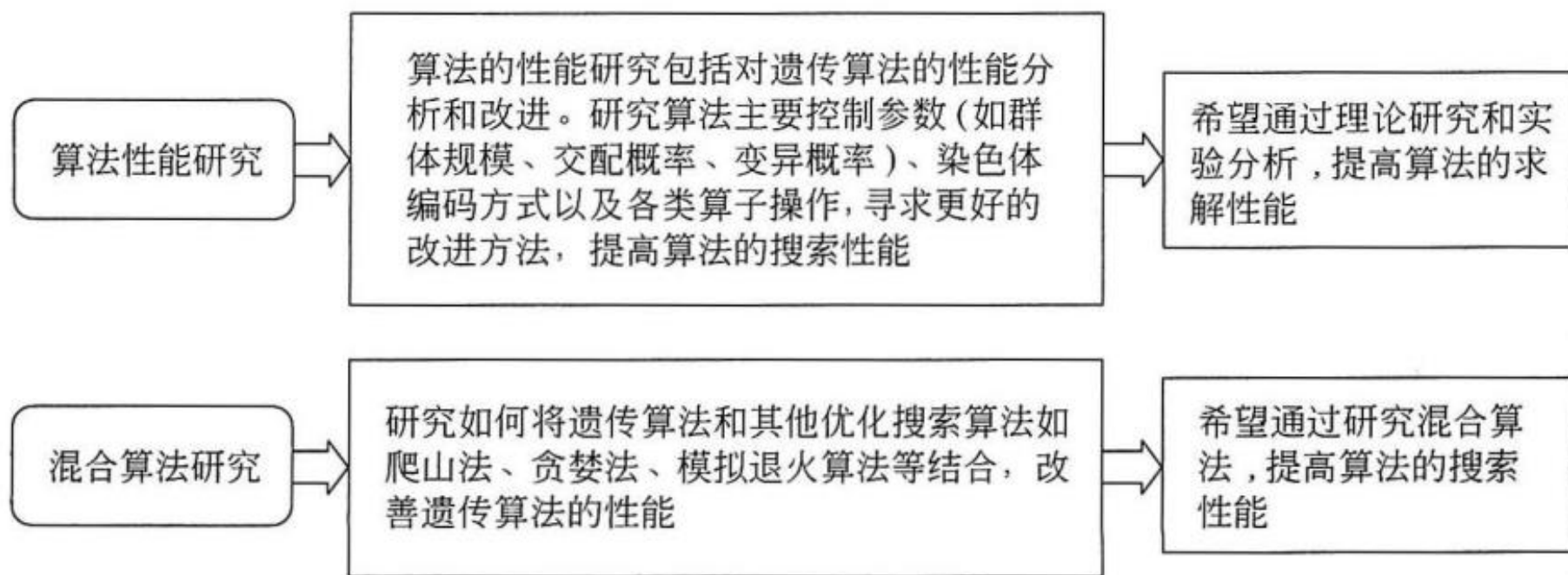
针对上述情况，需要在编码、适应度函数和遗传操作等设计中考虑抑制未成熟收敛的对策。这些对策包括：

- (1)提高变异概率。**在进化初始阶段，它可以加强遗传算法的随机搜索能力。
- (2)调整选择概率。**可以把选择概率本身也作为个体来进行优化，这就是所谓元遗传算法(meta GA)。
- (3)合适定标。**对适应度函数定标。

(4)维持群体中个体的多样性

- ①增加群体规模，但要考虑计算量增加的因素；
- ②实施局部化，把群体分割成若干子群体，每个子群体独立地进行选择操作，这样可使因出现不适当个体而产生未成熟收敛的现象局部化；
- ③实施单一化，把相同个体单一化，即不允许群体中有若干个相同个体出现；
- ④增大配对个体距离，配对选择一般是随机进行的，其中缺少对个体间相似度的判断，因此有可能使交叉结果未能产生新个体。

遗传算法研究内容



混合遗传算法

- √ 提高遗传算法求解问题的能力。
- 并行模拟退火遗传算法(Parallel Simulated Annealing and Genetic Algorithms , PSAGA)
- 贪婪遗传算法(Greedy Genetic Algorithm, GGA)
- 遗传比率切割算法(Genetic Ratio-Cut Algorithm, GRCA)
- 遗传爬山法(Genetic Hillclimbing Algorithm , GHA)
- 引入局部改善操作的混合遗传算法
- 免疫遗传算法(Immune Genetic Algorithm, IGA)

遗传算法研究内容

并行算法研究

随着大规模并行计算机的问世,并行计算成为现代计算机科学的重大发展方向。遗传算法自身具备的并行性适应这种发展趋势。通过研究实现遗传算法的并行计算,可以有效地提高算法的运行效率

希望通过研究,提高遗传算法的计算性能和运行效率,适应并行计算的发展趋势

算法应用研究

遗传算法最初是用于设计人工自适应系统的。自提出以来,算法的应用范围被快速地拓宽。目前对遗传算法应用的研究是一个热门的研究领域。各种研究相继出现:

- (1) 算法应用于函数优化、组合优化问题的研究;
- (2) 算法应用于机器学习的分类系统和神经网络的研究;
- (3) 算法应用于图像处理和模式识别的研究;

⋮

希望通过研究,不断拓展遗传算法应用于各种不同领域的有效性和通用性

并行遗传算法

√ 算法从整体的流程上看仍然是串行的，但是算法运行过程中对每个染色体的处理却是具有潜在的并行性。

■ 标准型并行方法(**Standard Parallel Approach**)

没有根本改变遗传算法整体上的串行计算结构，只是在算法的某些操作中引入并行计算技术，这些操作包括适应值计算、选择操作、交叉操作、变异操作等

■ 分解型并行方法(**Decomposition Parallel Approach**)

将整个群体分解成几个子群体，各个子群体分配到不同的计算资源上分别独立地使用原有的遗传算法进行进化

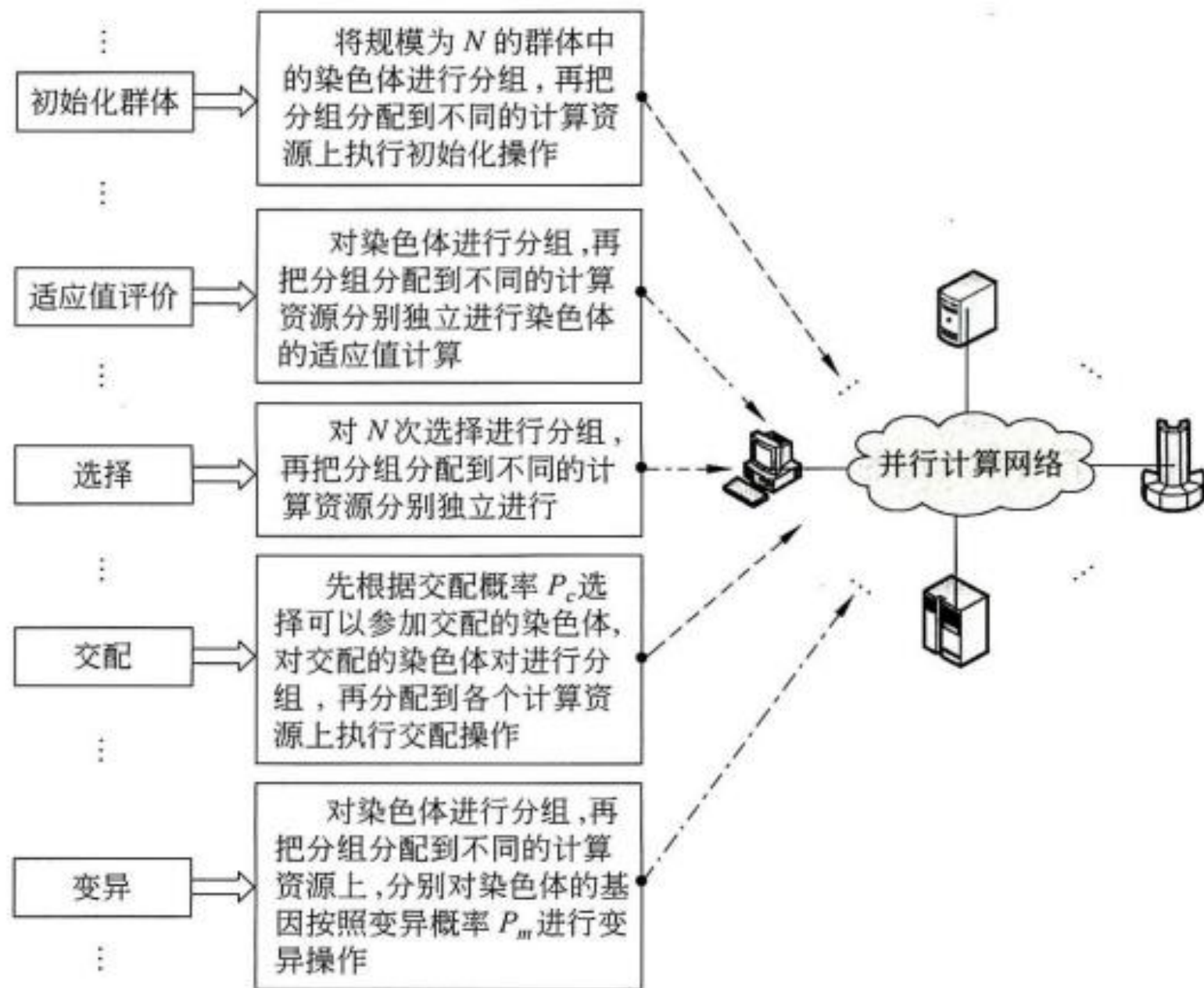


图 4.11 标准型并行方法简单示意图

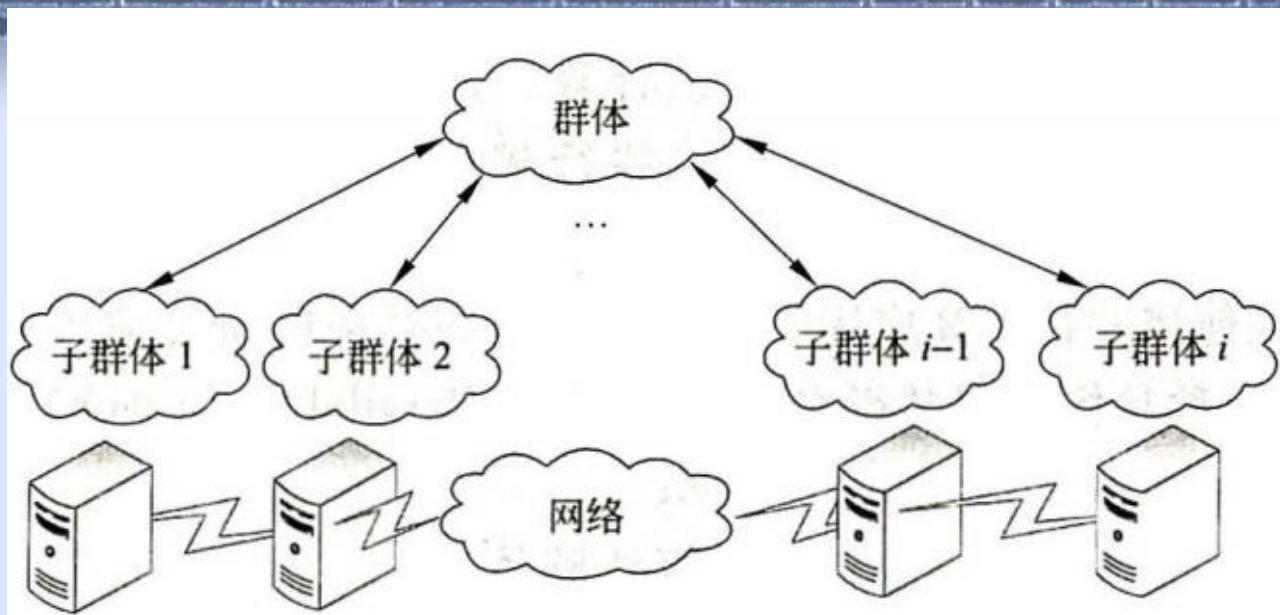


图 4.12 分解型并行方法简单示意图

- 分解型并行方法这种思想更贴近于自然界的生物进化系统。由于地域的限制，分布在不同地域的同一种生物的进化过程是不相同的，最终导致出现各种不同的物种。不同物种适应环境的能力不尽相同。
- 同样地，独立进行进化的各个子群体在各个阶段的进化程度也是不同的。

- 分解型并行方法要求每隔一定的进化代数需要对各个子群体的进化结果信息进行交换。
- 在分解型并行遗传算法中，各个子群体的信息交换是一个重要的操作。对于子群体之间如何交换进化信息，需要解决表 4.8 中列出的几个重要问题。

表 4.8 子群体进化信息交换问题

问题	对 应 的 定 义
交换的时间	每隔多少个进化代数实行信息交换
交换的方式	每次参与信息交换的子群体如何确定， 每个子群体和其他哪些子群体进行交换
交换的内容	可以是用于子群体之间适应值最大的最优染色体 取代参与交换的子群体的最优染色体， 也可以是交换子群体的部分较优染色体等